ASSIGNMENT 2 – TASK 4

INTER-PROCESS COMMUNICATION: PIPE AND SHARED MEMORY

1. Introduction and Problem Context

Task 4 describes a weather-station data pipeline. One process reads sensor values and another analyses them. The processes must exchange data continuously without losing readings. The assignment requires an implementation using a pipe and a second implementation using shared memory or a message queue. It also asks how end-of-data is signalled, how the communication models differ, which method copies data, which requires extra synchronisation, and how transfer time changes for 1 MB, 10 MB and 100 MB.

This report selects POSIX shared memory with process-shared semaphores as the second method. The pipe example uses the exact six sensor readings supplied in the question. The transfer benchmark compares both IPC methods using the same generated payload sizes and validates the received bytes with a checksum.

2. Aim and Objectives

Aim: To implement and compare pipe-based message passing and shared-memory communication between processes, and to evaluate their data-copying, synchronisation and transfer-time characteristics.

Objective	Work included
Pipe communication	Complete the supplied TODO, send integer readings from parent to child and detect EOF.
Shared-memory communication	Create a shared memory region mapped by two processes and coordinate producer/consumer access with semaphores.
End-of-data handling	Explain that read() returns 0 after all pipe write descriptors are closed and buffered data is consumed.
Growth investigation	Measure transfers of 1 MiB, 10 MiB and 100 MiB with both IPC methods.
Evaluation	Compare copying, synchronisation, timing, memory use and design trade-offs.

3. Pipe-Based Communication

A pipe is a unidirectional byte stream maintained by the operating system. The producer writes bytes to the write end, and the consumer reads bytes from the read end. The file descriptors are inherited across fork(), allowing the parent and child to communicate.

3.1 Supplied sensor readings

The assignment provides the integer readings 31, 33, 36, 34, 41 and 38. The parent writes each integer to the pipe. The child reads integers until read() returns 0, then calculates the reading count, average and maximum.

Expected output given in the assignment:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

3.2 How the pipe signals end of data

The parent closes the pipe's write descriptor after sending all readings. The child continues reading any bytes already buffered in the pipe. Once the buffer is empty and every open descriptor referring to the pipe's write end has been closed, read() returns 0. That zero return is EOF and tells the child there will be no more data. The parent then waits for the child using waitpid().

3.3 Why removing close(fd[1]) causes the child not to finish

If the parent does not close its write descriptor, the kernel still sees a possible writer. After the six readings have been consumed, the child blocks in read() waiting for additional bytes rather than receiving EOF. The parent may then block in waitpid() because the child has not terminated. This is why every unused pipe end must be closed in each process.

4. Shared-Memory Communication with Semaphores

Shared memory maps the same physical memory region into the address spaces of both processes. The producer writes sensor data into the shared region, and the consumer reads it directly from that region. After the mapping is established, the payload does not need to be copied through a kernel pipe buffer for each transfer.

The implementation uses POSIX shm_open(), ftruncate() and mmap() to create and map a shared-memory object. Two unnamed semaphores are stored inside the shared region and initialized with pshared=1 so that they are process-shared. The empty semaphore indicates that the shared slot is available for the producer; the full semaphore indicates that the consumer has data to read.

The benchmark uses a single shared buffer of 256 KiB. The producer waits for the slot to become empty, fills it with the next data chunk, and posts the full semaphore. The child waits for full, processes the chunk, and posts empty. After the final chunk, the producer sets an explicit finished flag and signals the consumer. This is a simple producer-consumer protocol.

5. Message Passing and Shared Memory: Comparison

Aspect	Pipe (message passing)	Shared memory
Communication model	Processes exchange a byte stream through a kernel-managed pipe buffer.	Processes read and write a common mapped memory region.
Payload copying	Typically copied from producer user space into the kernel pipe buffer and from the kernel buffer into consumer user space.	Payload can be accessed in the shared mapping without per-message copies between process address spaces.
Synchronisation	The pipe manages buffering and blocking read/write behaviour; descriptor closure provides EOF.	The application must coordinate access, here using process-shared semaphores.
Setup	pipe() and fork() establish descriptors for related processes.	Create/open shared object, size it, map it, initialize synchronisation objects and share its name/fd.

Aspect	Pipe (message passing)	Shared memory
Data access	Read and write through file descriptors.	Load/store through the shared mapping.
Typical trade-off	Simple and convenient for streams; kernel-mediated copies and buffer limits.	Efficient for larger repeated transfers; coordination and lifecycle management are more complex.

6. Data Copying and Synchronisation Analysis

For a conventional pipe, the producer's write transfers data from its user buffer to a kernel-managed pipe buffer, and the consumer's read copies data from that buffer into its user buffer. This is commonly described as two payload copies through the kernel. A shared-memory mapping avoids those per-transfer payload copies because both processes access the same mapped pages. Setup calls, page mapping, semaphore operations and cache-coherence traffic still have costs; shared memory is not literally overhead-free.

A pipe naturally provides blocking and stream semantics. Shared memory does not by itself define when a producer has finished writing or when a consumer may safely read. Without synchronisation, the consumer could read partially updated data or the producer could overwrite data before it is consumed. The semaphores prevent those races by enforcing the empty/full hand-off.

7. Transfer-Time Measurement Method

The supplied ipc_benchmark.c program transfers 1 MiB, 10 MiB and 100 MiB from a parent producer to a child consumer using both a pipe and shared memory with semaphores. The same deterministic byte pattern is used for both methods. The receiver calculates a checksum, and the program validates it against the expected checksum so that the transfer is not merely timed without checking the data.

Each case is repeated three times and the median elapsed time is printed. The timed interval includes producing and transferring the bytes and waiting for the consumer to finish; IPC object creation and fork setup occur before the timer starts. The benchmark labels sizes in MiB, where 1 MiB = 1,048,576 bytes. This is the binary interpretation of the requested 1 MB, 10 MB and 100 MB sizes.

Data size	Pipe median time (ms)	Shared-memory median time (ms)	Checksum validation
1 MiB	Record local result	Record local result	PASS required
10 MiB	Record local result	Record local result	PASS required
100 MiB	Record local result	Record local result	PASS required

No machine-specific transfer times are pre-filled in this report. Run the benchmark on the Ubuntu/WSL system used for submission and copy the measured medians from ipc_benchmark_results.txt into the table. Do not use another computer's timing values as your own.

8. How to Interpret the Measurements

A pipe may perform well for small or moderate transfers because its interface is simple and the kernel manages buffering. For large repeated transfers, the shared-memory approach may benefit from avoiding payload copies, but semaphore hand-offs and chunk size influence the result. In this implementation, each 256 KiB chunk requires producer/consumer semaphore coordination. The actual winner depends on the operating system, CPU, memory system, scheduler, transfer size and implementation details; the measurements should determine the recommendation.

For a fair comparison, both methods use the same byte pattern, transfer direction, total data size and checksum validation. The median of repeated runs is reported to reduce the influence of one unusually fast or slow run. Record compiler, Linux distribution, kernel version and machine information alongside the measurements.

9. Alternative Concept and Complexity Analysis

The chosen alternative to a pipe is shared memory coordinated by semaphores. A producer and consumer can share a ring buffer or a single-slot buffer. The producer waits for an available slot, writes a data chunk and signals that it is full. The consumer waits for a full slot, processes the chunk and signals that it is empty. The benchmark uses a single-slot buffer to keep the protocol simple.

Measure	Pipe	Shared memory + semaphores
Payload work for n bytes	O(n) bytes are written and read; pipe copies payload through kernel buffer.	O(n) bytes are written and read in the shared region; no per-transfer payload copying between address spaces.
Coordination operations	System calls for read/write; with chunk size B, approximately O(n/B) transfer calls plus EOF/close.	Semaphore hand-offs per chunk; approximately O(n/B) producer/consumer hand-offs.
Buffer space	Kernel pipe buffer plus producer/consumer user buffers; bounded by OS pipe capacity.	O(B) shared buffer in this single-slot design, plus mapping/control metadata.
Setup cost	pipe/fork and descriptor management.	Shared-memory creation, sizing, mapping and semaphore initialization.

Both approaches must handle n bytes, so the amount of payload processing is O(n). The important difference is the constant-factor cost: a pipe copies payload into and out of a kernel buffer, while shared memory avoids those per-transfer payload copies. With chunk size B, both approaches perform coordination in proportion to the number of chunks, approximately O(n/B). Shared memory adds explicit semaphore synchronisation and lifecycle management. The memory requirement for the single-slot shared-memory buffer is O(B), independent of total transfer size n; the pipe also uses bounded kernel buffering plus fixed user buffers.

10. Design Recommendation

For a small, simple sensor stream with a single producer and consumer, a pipe is a strong choice because it is straightforward, provides blocking stream semantics and signals EOF when the writer closes the final write descriptor. It is easy to debug and requires little application-level synchronisation.

For large or repeated transfers where throughput is important and the processes need to access the same data region, shared memory with semaphores may be preferable because it avoids per-transfer payload copying. However, the application must correctly implement producer/consumer synchronisation, handle process failure and clean up shared-memory objects. The final recommendation for the measured workload should be based on the local transfer-time table and the complexity/robustness requirements.

11. Conclusion

The pipe implementation demonstrates process communication through a kernel-managed byte stream. Closing the parent's write descriptor is essential because it allows the reader to observe EOF after buffered data is consumed. The second implementation demonstrates shared-memory communication using a process-shared semaphore protocol. Pipes offer a simple stream abstraction with kernel-managed buffering, while shared memory can avoid payload copies but requires explicit synchronisation. The benchmark compares both methods for increasing payload sizes and validates the transferred data with checksums. A justified choice should combine the measured local timings with implementation complexity, data volume and reliability requirements.

12. Execution Procedure and Evidence

1. Keep pipe_readings.c, ipc_benchmark.c and run_task4.sh in the same Task04 directory.
2. Open Ubuntu/WSL in that directory.
3. Run chmod +x run_task4.sh.
4. Run ./run_task4.sh.
5. The script compiles both programs, runs the supplied six-reading example, then benchmarks pipe and shared memory at 1, 10 and 100 MiB.
6. Save pipe_readings_output.txt, ipc_benchmark_results.txt and environment.txt.
7. Capture a terminal screenshot showing the expected readings output and all six checksum-validated transfer cases.

Evidence expected by the assignment: two working IPC implementations, measured transfer times for 1 MB, 10 MB and 100 MB, and a justified choice of method.

Source alignment: This report addresses Q4 – Inter-Process Communication, 6 Marks, K5 – Evaluate, on pages 14–15 of the supplied Assignment 2 question paper.